

Thiết kế BRAM

1. Tổng quan về BRAM (Block RAM)

BRAM (Block Random Access Memory) là các khối bộ nhớ nhúng được tích hợp sẵn bên trong các dòng FPGA. Khác với bộ nhớ phân tán (Distributed RAM) được tạo ra từ các LUTs, BRAM mang lại hiệu suất cao hơn, tiết kiệm tài nguyên logic và hỗ trợ dung lượng lưu trữ lớn hơn.

Trong bài thực hành này, thiết kế tập trung vào việc mô hình hóa một **Single-Port BRAM** (BRAM Đơn cổng) đồng bộ với xung nhịp (Clock), cho phép thực hiện thao tác Đọc (Read) hoặc Ghi (Write) tại một thời điểm thông qua một địa chỉ duy nhất.

2. Các chế độ hoạt động (Operating Modes)

Điểm cốt lõi của thiết kế này là việc khảo sát hành vi của ngõ ra dữ liệu (dout) khi xảy ra thao tác Ghi (we = 1). Mạch hỗ trợ 3 chế độ hoạt động tiêu chuẩn:

- **Chế độ 0 - Write First (Ghi trước):** Ngõ ra dout lập tức phản ánh dữ liệu mới (din) đang được ghi vào địa chỉ hiện tại. Chế độ này còn được gọi là "Trong suốt" (Transparent).
- **Chế độ 1 - Read First (Đọc trước):** Ngõ ra dout xuất ra dữ liệu cũ (đã tồn tại sẵn trong ô nhớ) trước khi dữ liệu mới (din) đè lên ô nhớ đó.
- **Chế độ 2 - No Change (Không thay đổi):** Ngõ ra dout giữ nguyên trạng thái của chu kỳ trước đó, hoàn toàn phớt lờ thao tác ghi đang diễn ra.

3. Code verilog

```
module bram_mode #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 4,
    parameter MODE = 0
    // 0 -> write first
    // 1 -> read first
    // 2 -> no change
)(
    input clk,
    input en, // BRAM Enable
    input we, // Write Enable
    input [ADDR_WIDTH-1:0] addr,
    input [DATA_WIDTH-1:0] din,
    output reg [DATA_WIDTH-1:0] dout
);

localparam DEPTH = (1 << ADDR_WIDTH);

(*ram_style = "block" *)
reg [DATA_WIDTH-1:0] mem[0:DEPTH-1];
```

```

generate
  if (MODE == 0) begin : write_first
    always @(posedge clk) begin
      if (en) begin
        if (we) begin
          mem[addr] <= din;
          dout <= din;
        end
        else
          dout <= mem[addr];
        end
      end
    end
  end
  else if (MODE == 1) begin : read_first
    always @(posedge clk) begin
      if (en) begin
        if (we)
          mem[addr] <= din;
          dout <= mem[addr];
        end
      end
    end
  end
  else if (MODE == 2) begin : no_change
    always @(posedge clk) begin
      if (en) begin
        if (we)
          mem[addr] <= din;
        else
          dout <= mem[addr];
        end
      end
    end
  end
endgenerate
endmodule

```

4. Testbench

Để kiểm chứng hoạt động của 3 chế độ, Testbench được thiết kế để khởi tạo cùng lúc 3 khối BRAM (đại diện cho 3 Mode). Kịch bản bao gồm các thao tác: ghi dữ liệu ban đầu, đọc dữ liệu, và quan trọng nhất là thao tác ghi đè để phân biệt rõ ràng ngõ ra của 3 chế độ.

```

module tb_bram_mode_advanced;

  parameter DATA_WIDTH = 8;
  parameter ADDR_WIDTH = 4;

```

```

reg clk;
reg en;
reg we;
reg [ADDR_WIDTH-1:0] addr;
reg [DATA_WIDTH-1:0] din;

wire [DATA_WIDTH-1:0] dout_wf;
wire [DATA_WIDTH-1:0] dout_rf;
wire [DATA_WIDTH-1:0] dout_nc;

// Khởi tạo 3 BRAM (Write First, Read First, No Change)
bram_mode #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH),
.MODE(0)) dut_wf (.clk(clk), .en(en), .we(we), .addr(addr), .din(din), .dout(dout_wf));
bram_mode #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH),
.MODE(1)) dut_rf (.clk(clk), .en(en), .we(we), .addr(addr), .din(din), .dout(dout_rf));
bram_mode #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH),
.MODE(2)) dut_nc (.clk(clk), .en(en), .we(we), .addr(addr), .din(din), .dout(dout_nc));

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    // Khởi tạo
    en = 0; we = 0; addr = 0; din = 0;
    #50;

    // Ghi dữ liệu
    @(negedge clk);
    en = 1; we = 1; addr = 4'h1; din = 8'hAA;

    @(negedge clk);
    en = 1; we = 1; addr = 4'h2; din = 8'hBB;

    // Đọc địa chỉ 2
    @(negedge clk);
    en = 1; we = 0; addr = 4'h2; din = 8'h00;

    // Test
    @(negedge clk);
    en = 1; we = 1; addr = 4'h1; din = 8'hCC;
    // Kỳ vọng: wf=CC, rf=AA, nc=BB

    // Ghi liên tiếp vào cùng 1 địa chỉ (Back-to-back write)
    @(negedge clk);

```

```

en = 1; we = 1; addr = 4'h1; din = 8'hDD;

// Tắt Enable (en = 0) nhưng vẫn bật Write (we = 1)
@(negedge clk);
en = 0; we = 1; addr = 4'h5; din = 8'hFF;

// Đọc kiểm tra lại địa chỉ 5 (Xác nhận Case 2 an toàn)
@(negedge clk);
en = 1; we = 0; addr = 4'h5; din = 8'h00;

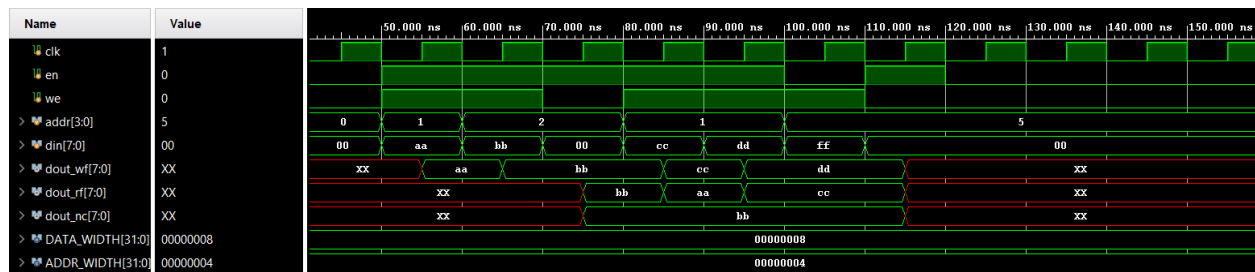
// Kết thúc
@(negedge clk);
en = 0;

#50;
$finish;
end

endmodule

```

5. Kết quả mô phỏng



- Ghi:
 - o 55ns: en = 1, we = 1: Ghi aa vào addr 1
 - o 65ns: en = 1, we = 1: Ghi bb vào addr 2
- Đọc:
 - o 75ns: en = 1, we = 0: Không ghi 00 mà đọc bb tại addr 2 → Đồng bộ ngõ ra của 3 BRAM
- Ghi đè:
 - o 85ns: en = 1, we = 1: Ghi cc vào addr 1
 - wf (bram ghi): hiển thị dữ liệu mới đã ghi
 - rf (bram đọc): đọc giá trị trước đó (aa)
 - nc (bram giữ): giữ nguyên trạng thái
- Kiểm tra tín hiệu enable và write enable:
 - o 105ns: en = 0 → 3 bram đều giữ giá trị trước đó
 - o 115ns: we = 0, en = 1 → không ghi giá trị vào add5, tuy nhiên trước đó addr 5 không có giá trị, nên cả 3 thanh ghi đều hiện XX